

SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-02 pairs with TB-M3-02

Top-Level Integration Lab

Wire elevator, arbiter, clock_gate, and axiom_shim into a single tt_um compliant top.

Learning objectives

- Translate the tt_um pinout contract into named-port bindings inside top.v.
- Use valid/ready handshakes between elevator and arbiter without deadlocking either side.
- Correctly partition reset domains — elevator_rst_n, arbiter_rst_n, axiom_rst_n — and understand why they must NOT be tied together.
- Run the integration smoke test and read its FSM trace to confirm all four sub-modules wake up in the right order.

Prep checklist

- ☐ SG-M3-01 complete, first LibreLane run finished (even if DRC failed).
- ☐ You have re-read TB-M3-02 — especially the pinout allocation table.
- ☐ Your M2 elevator.v compiles standalone (run `make lint` in src/).

Step 1 · Read the pinout contract

ChipFoundry shuttle tiles expose exactly 8 input pins, 8 output pins, and 8 bidirectional pins. They are named `ui\_in`, `uo\_out`, and `uio`. Every top.v in the cohort must match this contract — the pad ring is fixed.

```
module tt_um_silicon_dreams (  
    input  wire [7:0] ui_in,    // dedicated inputs  
    output wire [7:0] uo_out,   // dedicated outputs  
    input  wire [7:0] uio_in,   // bidirectional inputs  
    output wire [7:0] uio_out,  // bidirectional outputs  
    output wire [7:0] uio_oe,   // bidirectional output-enable
```

```

input wire    ena,      // always 1 when the design is selected
input wire    clk,
input wire    rst_n     // active-LOW reset (important!)
);

```

E8873C

rst_n is ACTIVE-LOW. Your hardened M2 elevator already handles this, but arbiter, clock_gate, and axiom_shim may need a polarity flip before being fed their own reset signals.

Step 2 · Fill in the M3 pinout allocation

From TB-M3-02 Table 2.1:

ui_in[0]	request_strobe → elevator
ui_in[4:1]	requested_floor[3:0] → elevator
ui_in[5]	priority_override_req → arbiter
ui_in[6]	axiom_enable → axiom_shim
ui_in[7]	debug_probe_select
uo_out[3:0]	current_floor[3:0] ← elevator
uo_out[6:4]	state[2:0] ← elevator
uo_out[7]	door_open ← elevator
uio[0]	fault_inject_enable (input; held in M3 too)
uio[1]	axiom_misbehaviour_led (output)
uio[2]	arbiter_grant_elevator (output, debug)
uio[3]	arbiter_grant_axiom (output, debug)
uio[7:4]	reserved / error + clock-gate status

Step 3 · Map ports, not wires

Always use NAMED port connections. Positional connections break horribly during integration and the course lint will reject them.

```

// GOOD
elevator u_elevator (
    .clk          (clk),
    .rst_n        (elevator_rst_n),
    .request_strobe (request_strobe),
    .requested_floor (requested_floor),
    .fault_inject_en (fault_inject_enable),
    .current_floor  (current_floor),
    .state         (elevator_state),

```

```

        .door_open      (door_open),
        .error_led      (elevator_error_led)
    );

    // BAD
    elevator u_elevator (clk, elevator_rst_n, ui_in[0], ...);

```

Step 4 · Reset domain partitioning

Every sub-module gets its OWN reset signal derived from `rst_n`. Do NOT tie them together. The reasons:

- `elevator_rst_n` is held low for 4 cycles after `rst_n` deasserts — this lets the elevator state machine arrive in IDLE deterministically.
- `arbiter_rst_n` is held low for 6 cycles — the arbiter has a 3-deep priority queue that needs an extra two cycles to flush.
- `axiom_rst_n` is INVERTED (active-HIGH for `axiom_blackbox`). See TB-M3-05 defensive rule #1.

```

// reset_synchroniser.v pattern – used three times in top.v
reset_synchroniser #(.HOLD_CYCLES(4)) u_rst_elev (
    .clk      (clk),
    .rst_n    (rst_n),
    .rst_n_sync (elevator_rst_n)
);

reset_synchroniser #(.HOLD_CYCLES(6)) u_rst_arb  (.clk(clk), .rst_n(rst_n),
.rst_n_sync(arbiter_rst_n));
assign axiom_rst = ~axiom_rst_n_sync; // polarity flip at the boundary

```

Step 5 · Valid/ready handshake between elevator and arbiter

The elevator emits a floor-request packet whenever a `request_strobe` lands. The arbiter must acknowledge before the elevator can emit another.

```

// Inside top.v
wire [3:0] req_payload;
wire      req_valid;
wire      req_ready;

elevator_req_port u_req_port (
    .clk      (clk),
    .rst_n    (elevator_rst_n),
    .strobe   (request_strobe),
    .floor    (requested_floor),
    .req_payload (req_payload),

```

```

    .req_valid    (req_valid),
    .req_ready    (req_ready)
);

arbiter u_arbiter (
    .clk          (clk),
    .rst_n        (arbiter_rst_n),
    .priority_req  (priority_override),
    .elev_req_valid (req_valid),
    .elev_req_ready (req_ready),
    .elev_req_payload(req_payload),
    .grant_elev    (grant_elev),
    .grant_axiom    (grant_axiom),
    .queue_nonempty (arbiter_queue_nonempty)
);

```

6B2E8F

Rule: `valid` MUST NOT depend combinationally on `ready`. If it does, the arbiter cannot assert ready without triggering another valid — infinite handshake loop. Register the valid signal on the elevator side.

Step 6 · Run the integration smoke test

```

cd test/top
make MODULE=test_top_smoke

```

```

# Expected output tail:
# == TOP INTEGRATION SMOKE =====
#  reset_release ..... PASS
#  elevator_wakes_in_IDLE ..... PASS
#  arbiter_grants_elevator_by_default ..... PASS
#  priority_override_grants_axiom ..... PASS
#  axiom_shim_clamps_misbehaviour ..... PASS
#  clock_gate_disables_on_idle ..... PASS
# == SUMMARY =====
#   8 pass, 0 fail, 0 skip

```

If `reset_release` fails, you have a reset-domain bug. If `elevator_wakes_in_IDLE` fails, you have an M2 regression — go back. If `arbiter_grants_elevator_by_default` fails, you have a priority logic bug (preview of SG-M3-03).

Troubleshooting

`Error: ui_in unused`	You forgot to wire ui_in[5] (priority_override_req). Lint will flag this.
`Error: uio_oe missing`	Every uio pin you drive must set its uio_oe bit. See pinout doc.
Smoke test hangs	Likely a handshake deadlock. Add `\$monitor(req_valid, req_ready)` and re-run — look for both stuck at X or at 1.
`axiom_blackbox not found` during synth	axiom_blackbox.v is a BEHAVIOURAL stub only. It is bound by the grader at P&R time. Do not try to flatten it.

Reflection

- Draw top.v as a box diagram with five boxes (elevator, arbiter, clock_gate, axiom_shim, reset_syncs) and label every connecting wire. Save this diagram as `docs/top-block-diagram.png`.
- Confirm all six smoke tests pass. If even one fails, do NOT proceed to SG-M3-03.